



SDEV 2451

Full Stack Web Development

Global State Management and Authentication - 1

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments
- No Cheating
- Be a good classmate
- Don't waste your time
- Show up to class

Agenda

On the right is what we will cover today.

- DRF `PageNumberPagination`
- Applying Pagination to a ViewSet
- Updating the API Layer
- Updating the Data-Fetching Hook
- Adding Pagination Controls
- The React Context API
- The `usePagination` Hook
- Extracting Pagination to a Component
- Scoping Providers at the Route Level
- A Second Context: Notifications
- The `Toast` Component
- The `useNotification` Hook
- Challenge — Page Size Selector
- Key Takeaways

DRF PageNumberPagination

DRF's built-in `PageNumberPagination` wraps list responses in a `{ count, next, previous, results }` envelope so the frontend can request pages on demand.

```
# books/pagination.py
from rest_framework.pagination import PageNumberPagination

class BookPagination(PageNumberPagination):
    page_size = 10
    page_size_query_param = "page_size"
    max_page_size = 100
```

DRF `PageNumberPagination` (continued)

- `page_size = 10` caps each response to 10 records; clients request page 2 with `?page=2`
- `page_size_query_param` lets clients override page size per request (e.g. `?page_size=25`)
- `max_page_size` prevents clients from bypassing pagination with an arbitrarily large request
- A dedicated `pagination.py` keeps the class reusable across multiple viewsets

Applying Pagination to a ViewSet

Setting `pagination_class` on an individual viewset applies pagination only to that endpoint, leaving all other viewsets unpaginated.

```
# books/views.py
from books.pagination import BookPagination

class BookViewSet(ModelViewSet):
    serializer_class = BookSerializer
    pagination_class = BookPagination

    def get_queryset(self):
        return Book.objects.all()

# ... detail actions unchanged ...
```

Applying Pagination to a ViewSet (continued)

- Only the list action is paginated — `@action(detail=True)` methods operate on a single object and bypass the paginator entirely
- Setting `pagination_class` per-viewset gives control; setting it globally in `settings.py` would paginate every endpoint
- The response shape changes from a plain array to `{ count, next, previous, results }` — the frontend must read `.results` for the list

Updating the API Layer

The fetch function needs to accept a page number and append it as a query parameter so DRF knows which page to return.

```
// src/api/books.js

export async function fetchBooks(page = 1) {
  const response = await fetch(`${BASE_URL}/books/?page=${page}`)
  if (!response.ok) throw new Error('Failed to fetch books')
  return response.json()
}
```

Updating the API Layer (continued)

- `page = 1` defaults to the first page so any call site that omits the argument continues to work
- `?page=${page}` is the only change — DRF's `PageNumberPagination` reads this parameter automatically
- The function still returns `response.json()` — the calling hook is responsible for reading the paginated envelope

Updating the Data-Fetching Hook

The hook accepts a page number, passes it to the fetch function via closure, and uses `useEffect` to invalidate the cache whenever the page changes.

```
// src/hooks/useBooks.js

export function useBooks(page = 1) {
  const queryClient = useQueryClient()

  const { data, isLoading } = useQuery({
    queryKey: ['books'],
    queryFn: () => fetchBooks(page),
  })

  useEffect(() => {
    queryClient.invalidateQueries({ queryKey: ['books'] })
  }, [page])

  return { books: data ?? { results: [], count: 0 }, isLoading }
}
```

Updating the Data-Fetching Hook (continued)

- `queryKey: [ 'books' ]` is stable — one cache entry, not one per page; pagination state lives in context, not in the cache key
- `queryFn: () ⇒ fetchBooks(page)` closes over the current page; after invalidation React Query refetches with the latest value
- `useEffect([page])` triggers `invalidateQueries` whenever the page changes, causing an immediate refetch
- Returning `{ results, count }` keeps the paginated response shape visible to consumers

Adding Pagination Controls

Local `useState` is the simplest way to start — the page number lives in the component and drives the hook.

```
function BooksPage() {
  const [page, setPage] = useState(1)
  const { books, isLoading } = useBooks(page)

  return (
    <div>
      {isLoading ? <Spinner /> : <BookList books={books.results} />}
      <div>
        <button disabled={page === 1} onClick={() => setPage(p => p - 1)}>
          Previous
        </button>
        <span>{books.count} total</span>
        <button onClick={() => setPage(p => p + 1)}>Next</button>
      </div>
    </div>
  )
}
```

Adding Pagination Controls (continued)

- `setPage(p ⇒ p - 1)` uses the functional updater form to avoid stale closure issues
- `books.count` displays the full dataset size without loading all records
- This works for a single component, but if another component also needs the current page, we would have to pass `page` and `setPage` down as props — known as **prop drilling**

The React Context API

The Context API solves prop drilling by creating a shared value that any descendant component can read or update directly — no props required.

```
// src/contexts/PaginationContext.jsx
import { createContext, useState } from 'react'

export const PaginationContext = createContext(null)

export function PaginationProvider({ pageSize = 10, children }) {
  const [page, setPage] = useState(1)
  const [totalCount, setTotalCount] = useState(0)
  const totalPages = Math.ceil(totalCount / pageSize) || 1

  return (
    <PaginationContext.Provider value={{
      page, totalCount, totalPages, setTotalCount,
      goToNext: () => setPage(p => Math.min(p + 1, totalPages)),
      goToPrevious: () => setPage(p => Math.max(p - 1, 1)),
    }}>
      {children}
    </PaginationContext.Provider>
  )
}
```

The React Context API (continued)

- `createContext(null)` — the `null` default is only reached when a consumer is outside all providers; the hook will guard against this
- `PaginationContext.Provider` makes `value` available to every descendant — no props required
- `totalPages = Math.ceil(totalCount / pageSize) || 1` — the `|| 1` prevents a zero before the first fetch
- Navigation functions (`goToNext` , `goToPrevious`) close over the latest `totalPages` so the page clamp is always current
- `pageSize` as a prop lets the same provider serve lists with different sizes

The usePagination Hook

Wrapping `useContext` in a dedicated hook adds a null guard and lets us compute derived booleans in one place rather than in every consumer.

```
// src/hooks/usePagination.js
import { useContext } from 'react'
import { PaginationContext } from '../contexts/PaginationContext'

export function usePagination() {
  const context = useContext(PaginationContext)
  if (!context) {
    throw new Error('usePagination must be used inside a PaginationProvider')
  }
  return {
    ...context,
    hasNext: context.page < context.totalPages,
    hasPrevious: context.page > 1,
  }
}
```

The `usePagination` Hook (continued)

- `if (!context)` converts the `null` default into a descriptive error — far more useful than a cryptic `TypeError: Cannot destructure property 'page' of null`
- `hasNext` and `hasPrevious` are computed in the hook, not stored in the context — derived values belong in the consumer interface
- Components call `usePagination()` and never touch `useContext` or `PaginationContext` directly; if the context shape changes, only this file needs updating

Extracting Pagination to a Component

Once pagination state lives in context, the UI controls can be their own component — zero props from the parent.

```
// src/components/BooksPagination.jsx
import { usePagination } from '../hooks/usePagination'

function BooksPagination() {
  const {
    page, totalPages, totalCount,
    hasNext, hasPrevious,
    goToNext, goToPrevious,
  } = usePagination()

  return (
    <div>
      <button disabled={!hasPrevious} onClick={goToPrevious}>Previous</button>
      <span>Page {page} of {totalPages} • {totalCount} total</span>
      <button disabled={!hasNext} onClick={goToNext}>Next</button>
    </div>
  )
}
```

Extracting Pagination to a Component (continued)

- `BooksPagination` reads everything from context — its parent passes zero props
- `onClick={goToNext}` passes the function reference directly; the provider already encapsulates the page-clamping logic
- This component can be moved anywhere inside the `PaginationProvider` tree (header, footer, sidebar) without changing its code or its parent

Scoping Providers at the Route Level

Placing the provider on the route element scopes it to a single page and lets the page component call `usePagination` directly — no inner wrapper component needed.

```
// src/App.jsx
<Route
  path="/books"
  element={
    <PaginationProvider pageSize={10}>
      <BooksPage />
    </PaginationProvider>
  }
/>

// src/pages/BooksPage.jsx
function BooksPage() {
  const { page, setTotalCount } = usePagination()
  const { books, isLoading } = useBooks(page)

  useEffect(() => {
    if (books.count !== undefined) setTotalCount(books.count)
  }, [books.count, setTotalCount])
}
```



Scoping Providers at the Route Level (continued)

- The provider mounts when the route activates and unmounts when the user navigates away — pagination state resets automatically
- `BooksPage` can call `usePagination()` because `PaginationProvider` is already above it in the tree when React renders it
- `BooksPage` (writes `totalCount`) and `BooksPagination` (reads `totalPages`) are siblings — their shared state lives in the context above both; neither needs to pass props to the other

A Second Context: Notifications

A `NotificationContext` solves a different global-state problem — triggering a toast from any component without threading callbacks through props.

```
// src/contexts/NotificationContext.jsx
import { createContext, useState } from 'react'
import Toast from '../components/Toast'

export const NotificationContext = createContext(null)

export function NotificationProvider({ children }) {
  const [notification, setNotification] = useState(null)

  function showSuccess(message) { setNotification({ message, type: 'success' }) }
  function showError(message)   { setNotification({ message, type: 'error' }) }
  function hide()                { setNotification(null) }

  return (
    <NotificationContext.Provider value={{ showSuccess, showError, hide }}>
      <Toast notification={notification} hide={hide} />
      {children}
    </NotificationContext.Provider>
  )
}
```

A Second Context: Notifications (continued)

- `notification` is `null` or `{ message, type }` — a single piece of state controls whether a toast is visible
- `showSuccess`, `showError`, and `hide` are plain functions — no `useCallback` needed here
- The provider renders `<Toast>` itself — consumers just add `<NotificationProvider>` and get toasts for free
- `Toast` receives `notification` and `hide` as props rather than calling `useContext`, which avoids a circular import between `Toast` and `NotificationContext`

The Toast Component

Toast is a presentational component — it receives the current notification as props, auto-hides after a timeout, and can be dismissed manually.

```
// src/components/Toast.jsx
import { useEffect } from 'react'

function Toast({ notification, hide }) {
  useEffect(() => {
    if (!notification) return
    const timer = setTimeout(hide, 3000)
    return () => clearTimeout(timer)
  }, [notification, hide])

  if (!notification) return null

  const alertClass = notification.type === 'success'
    ? 'alert-success' : 'alert-error'

  return (
    <div className="toast toast-top toast-end">
      <div className={`alert ${alertClass}`}>
        <span>{notification.message}</span>
        <button onClick={hide}>X</button>
      </div>
    </div>
  )
}
```



The Toast Component (continued)

- `return null` when there is no notification — the React pattern for conditional rendering without an empty container
- The cleanup function `() ⇒ clearTimeout(timer)` cancels the previous timer if a second notification arrives before the old one auto-hides
- `Toast` has no import from `NotificationContext` — it only displays what it is given and calls back when dismissed

The useNotification Hook

The same pattern as `usePagination` — wrap `useContext`, add a null guard, and give every component a clean API to trigger notifications.

```
// src/hooks/useNotification.js
export function useNotification() {
  const context = useContext(NotificationContext)
  if (!context) throw new Error('useNotification must be used inside a NotificationProvider')
  return context // { showSuccess, showError, hide }
}

// Usage in any page component:
const { showSuccess, showError } = useNotification()

mutate(data, {
  onSuccess: () => { showSuccess('Post saved!'); navigate('/posts') },
  onError:    () => showError('Something went wrong. Please try again.'),
})
```

The `useNotification` Hook (continued)

- The hook returns only `{ showSuccess, showError, hide }` — `notification` state is an internal detail of the provider, not part of the public API
- The triggering component and the `Toast` display component are fully decoupled — neither knows the other exists
- `showSuccess` before `navigate( ... )` works because `Toast` is rendered inside `NotificationProvider`, which sits above the routes and survives route changes

Challenge — Page Size Selector

1. Add a page size selector

- Add a `<select>` with options for 5, 10, and 25 items per page
- Pass the selection to the fetch function as a second argument: `fetchBooks(page, pageSize)`
- Append `&page_size=${pageSize}` to the URL when provided
- Reset `page` back to 1 whenever `pageSize` changes

2. Show current page number

- The current controls show Previous / Next but no page indicator
- Add a `Page {page} of {totalPages}` label between the two buttons
- Derive `totalPages` from `Math.ceil(count / pageSize)`

Key Takeaways

- `PageNumberPagination` wraps list results in `{ count, next, previous, results }` automatically — no serializer changes required
- **Per-viewset** `pagination_class` applies pagination only where needed;
`@action(detail=True)` methods bypass it entirely
- **Stable** `queryKey` + `useEffect` **invalidation** separates pagination state from the React Query cache key
- **The Context API** eliminates prop drilling — `createContext` creates the value, `Provider` shares it, `useContext` reads it from any descendant
- **Derived values belong in the hook** — `hasNext` and `hasPrevious` are computed in `usePagination`, not stored in the context
- **Guard** `useContext` **with a null check** — a descriptive error beats a cryptic `TypeError` deep in the tree

WE ARE ESSENTIAL
TO ALBERTA





Example

Let's go do an example together

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS